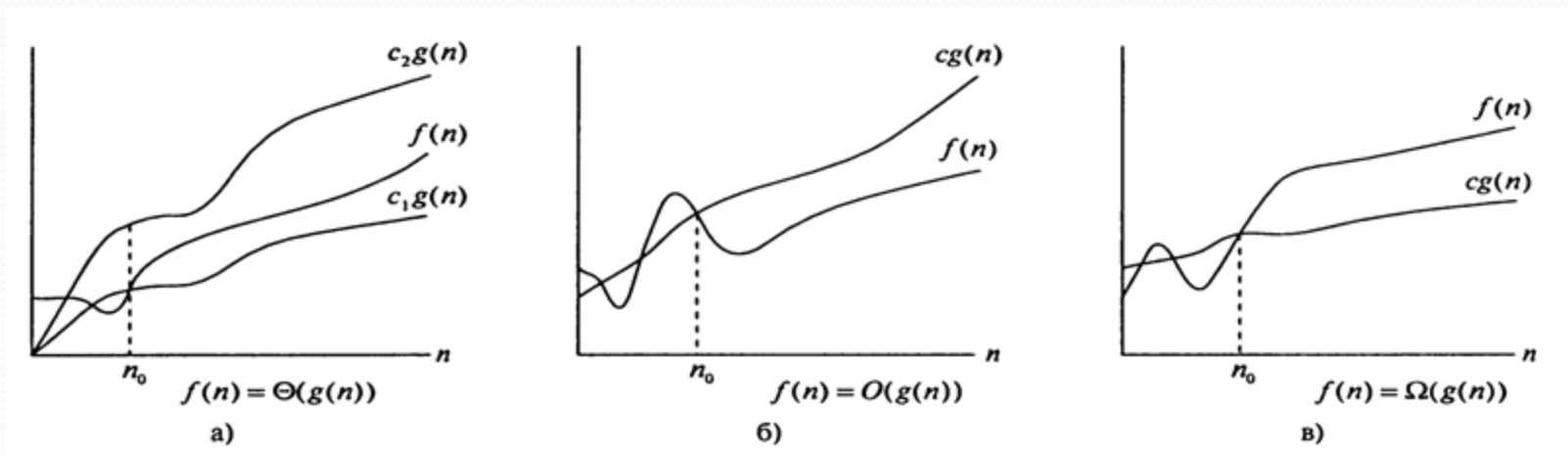


Классические алгоритмы и структуры данных

Временная сложность алгоритмов

Асимптотические обозначения



- Графические представления обозначений
 - а) $f(n) = \Theta(g(n))$ (тета)
 - б) $f(n) = O(g(n))$
 - в) $f(n) = \Omega(g(n))$ (омега)
- в каждой части рисунка в качестве n_0 используется минимально возможное значение, т.е. любое большее значение также сможет выполнить роль n_0

Асимптотические обозначения

- Пусть даны две функции $f(n)$ и $g(n)$ натурального аргумента n , значениями которых являются положительные действительные числа
- Говорят, что $f = O(g)$ (f растет быстрее g), если существует такая константа $c > 0$, что $f(n) \leq c \cdot g(n)$ для всех $n \in N$
- Обозначение $O()$ можно считать аналогом $\leq$
- Аналогом для $\geq$ и $=$ такие:
 - $f = \Omega(g)$ (омега, f растет не медленнее g , с точностью до константы) означает $g = O(f)$
 - $f = \Theta(g)$ (тета, f и g имеют одинаковый порядок роста) означает, что $f = O(g)$ и $g = O(f)$

Асимптотические обозначения

- Правила асимптотических замен:

1. Постоянные множители можно опускать. Например, $14 \cdot n^2$ (или $3 \cdot n^2 + 4 \cdot n + 5$) можно заменить на n^2

2. $n \cdot a$ растёт быстрее $n \cdot b$ для $a > b$. Например, в присутствии слагаемого n^2 можно пренебречь слагаемым n

3. Любая экспонента растёт быстрее любого многочлена (полинома). Например, 3^n растёт быстрее n^5

4. Любой полином растёт быстрее любого логарифма. Например, n (и даже $\sqrt{n}$) растёт быстрее $(\log n)^3$. Основание логарифма не указывается, поскольку замена основания приводит лишь к умножению логарифма на константу. По тем же причинам n^2 растёт быстрее $n \log n$

Временная сложность алгоритмов

	n	$n \cdot \log_2 n$	n^2	2^n	$n!$
n=10	<1с	<1с	<1с	<1с	<4с
n=30	<1с	<1с	<1с	18 мин.	10^{25} лет
n=50	<1с	<1с	<1с	36 лет	очень долго
n=100	<1с	<1с	<1с	10^{17} лет	очень долго
n=1000	<1с	<1с	1с	очень долго	очень долго
n=10000	<1с	<1с	2 мин.	очень долго	очень долго
n=100000	<1с	<1с	3 часа	очень долго	очень долго
n=1000000	<1с	<2с	12 дней	очень долго	очень долго

Время выполнения (округление вверх) разных алгоритмов для входных данных разного размера (для процессора, выполняющего миллион высокоуровневых команд в секунду). Если время больше 10^{25} – «очень долго»

Временная сложность алгоритмов

- $O(1)$ – время работы алгоритма (программы) постоянно
- $O(n)$ – время является линейным
- $O(n^2)$ – время является квадратичным
- $O(\log n)$ – время является логарифмическим

Алгоритмы поиска

Задача

- Подсчитать в одномерном/двумерном массиве количество элементов v
- Определить, есть ли в одномерном/двумерном массиве элемент v

Последовательный поиск

- Данная функция проверяет, находится ли число v среди элементов массива $a[1], a[1 + 1], \dots, a[n]$ путем последовательного сравнения с каждым элементом, начиная с начала
- Если по достижении последнего элемента нужное значение не найдено, то функция возвращает значение -1. Иначе она возвращает индекс элемента массива, содержащего искомое число

Последовательный поиск

Функция `Search(v)`

{Вход: массив $a[0..n-1]$, v – искомое значение}

{Выход: индекс искомого элемента, или -1 , если элемент не найден}

$i=0$

пока $(i < n)$ и $(a[i] \neq v)$: $i=i+1$

если $i \geq n$: вернуть i

иначе: вернуть -1

Последовательный поиск

```
void linearSearch(int arr[], int key) {  
    bool found = false;  
    for (int i = 0; i < SIZE; ++i)  
        if (arr[i] == key) {  
            std::cout << key << " found (#" << i  
                << ")" << std::endl;  
            found = true;  
        }  
    if (!found)  
        std::cout << key << " not found!" << std::endl;  
}
```

Исправить ошибки?

Последовательный поиск

- Последовательный поиск исследует n чисел при каждом неуспешном поиске и в среднем порядка $n/2$ чисел при каждом успешном поиске
- Если каждое число в массиве с равной вероятностью может быть объектом поиска, тогда $(1 + 2 + \dots + n) / n = (n + 1) / 2$ является средней ценой поиска
- $O(n)$

Задача

- Определить, есть ли в одномерном массиве элемент v (добавить барьерный элемент)

Барьерный поиск

$i=0$

пока $(i < n)$ и $(a[i] \neq v)$: $i=i+1$

$a[n] = v$

пока $(a[i] \neq v)$: $i=i+1$

если $i \neq n$: вернуть i

иначе: вернуть -1

Задача

- Определить, есть ли в одномерном массиве элемент v , если массив упорядочен по возрастанию/убыванию (бинарный/двоичный поиск)

Бинарный поиск

индекс	1	2	3	4	5	6	7	8	9
a	7	9	11	13	16	21	25	29	33

$v=11$

$l=1$ $r=9$ $m=5$

$a[5] \neq 11$

индекс	1	2	3	4
a	7	9	11	13

$l=1$ $r=4$ $m=2$

$a[2] \neq 11$

индекс	3	4
a	11	13

$l=3$ $r=4$ $m=3$

$a[3] = 11$

Бинарный поиск

Функция Search(v)

{Вход: отсортированный массив $a[1..n]$, v – искомый элемент}

{Выход: индекс искомого элемента в случае успешного поиска,
-1, если элемент не найден}

$l=1$

$r=n$

$pp=false$

пока $(r \geq l)$ и $(pp == false)$:

$m = (l+r)//2$ {целочисленное деление}

 если $a[m] == v$: $pp=true$

 иначе если $(v < a[m])$: $r=m-1$

 иначе $l=m+1$

если $pp == true$: вернуть m

иначе: вернуть -1

Бинарный поиск

```
int recursiveSearch(int arr[], int key, int left, int
right) {
    if left > right
        return -1;
    int middle = (left + right) / 2;
    if (arr[middle] == key)
        return middle;
    if (key < arr[middle])
        recursiveSearch(arr, key, left, middle - 1);
    else
        recursiveSearch(arr, key, middle + 1, right);
}
```

Утверждение. Бинарный поиск исследует не более $\lfloor \log_2 n \rfloor + 1$ чисел.

Доказательство. Пусть $T(n)$ – это количество сравнений, необходимое бинарному поиску в худшем случае для поиска элемента. Тогда из того, что алгоритм сводит поиск в массиве размера n к поиску в два раза меньшей части массива, непосредственно следует, что $T(n) \leq T(\lfloor n/2 \rfloor) + 1$, при $n \geq 2$ и $T(1) = 1$. При поиске в массиве размером n проверяется число посередине, затем производим поиск в части массива размером не более $\lfloor n/2 \rfloor$. Реальная цена может быть меньше этого значения, так как сравнение может закончиться успешно.

Предположим, что $n = 2^k$. Заметим, что $k = \log_2 n$. Тогда имеем: рекурсию еще проще раскрыть, чем в предыдущем случае:

$$\begin{aligned} T(2^k) &= T(2^{k-1}) + 1 = T(2^{k-2}) + 1 + 1 = T(2^{k-3}) + 3 = \dots = \\ &= T(2^0) + k = k + 1. \text{ Доказано.} \end{aligned}$$